Python Data Engines: State of the Union 2026

Every Python data engine looks impressive when you throw a 64-core machine at it. That is not this talk.

About me

Mimoune Djouallah

Microsoft · Fabric CAT team

Brisbane, Australia

Ex construction planner

linkedin.com/in/mimounedjouallah

A personal project. My own opinions.

Agenda

01 What changed since 2025

02 Iceberg

03 The engines

04 What does “fast” mean?

05 Real-life use case

06 What I learned

One job out of many

Read and serve

Dashboards and BI queries on tables that already exist.

Explore

Ad-hoc questions in a notebook.

Complex calculations

Science, simulations, ML features.

Streaming

Events in, results out, all the time.

Embedded

An engine inside an app or a browser.

Data engineering

Read raw files, transform, MERGE, write tables. Every day, unattended.

This talk: data engineering only.

What changed since 2025

SQL

Polars SQL went from a side feature to all 99 TPC-DS-like queries.

Given enough time, every DataFrame library becomes a database.

Iceberg

Iceberg is the new baseline.

Last year most of these engines couldn’t write a table; now they all write Iceberg.

AI

It’s very good at reproducing bugs.

Python engines aren’t niche anymore. They’re mainstream, and a bit boring.

The DataFrame API used to be a pain point. For a couple of months now, I don’t care.

02

Iceberg

All engines are equal, as long as they speak the same spec.

How we got here

1990s

The warehouse

Storage and compute in one product. The engine owns the data; changing engine is a migration.

2006

Big data: Hadoop

Files spread over a cluster's disks. The data is open, but you need the cluster to read it.

2010s

The cloud warehouse

Elastic compute, but the tables live inside one vendor.

2017

Open table formats

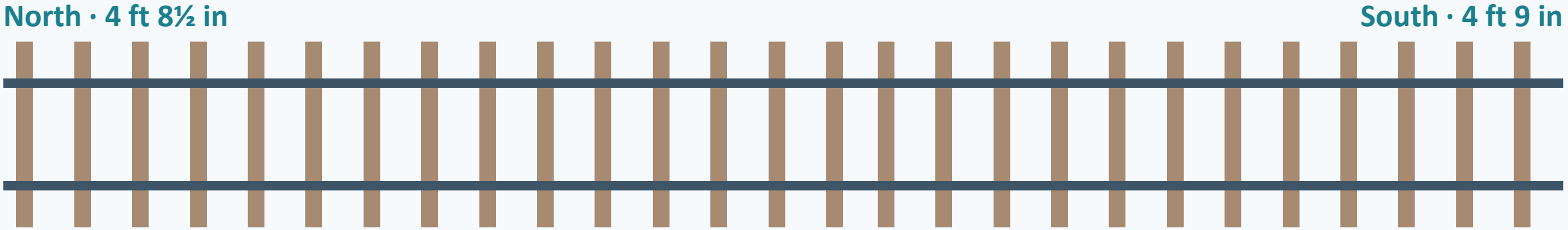
Iceberg (Netflix), Hudi, Delta: tables in object storage, but writing one needed a Java cluster (Spark, Trino).

Now

The open lakehouse

A REST catalog, and table libraries in Rust, C++ and Python: a library in a Python process writes the same tables as a cluster.

Break of gauge

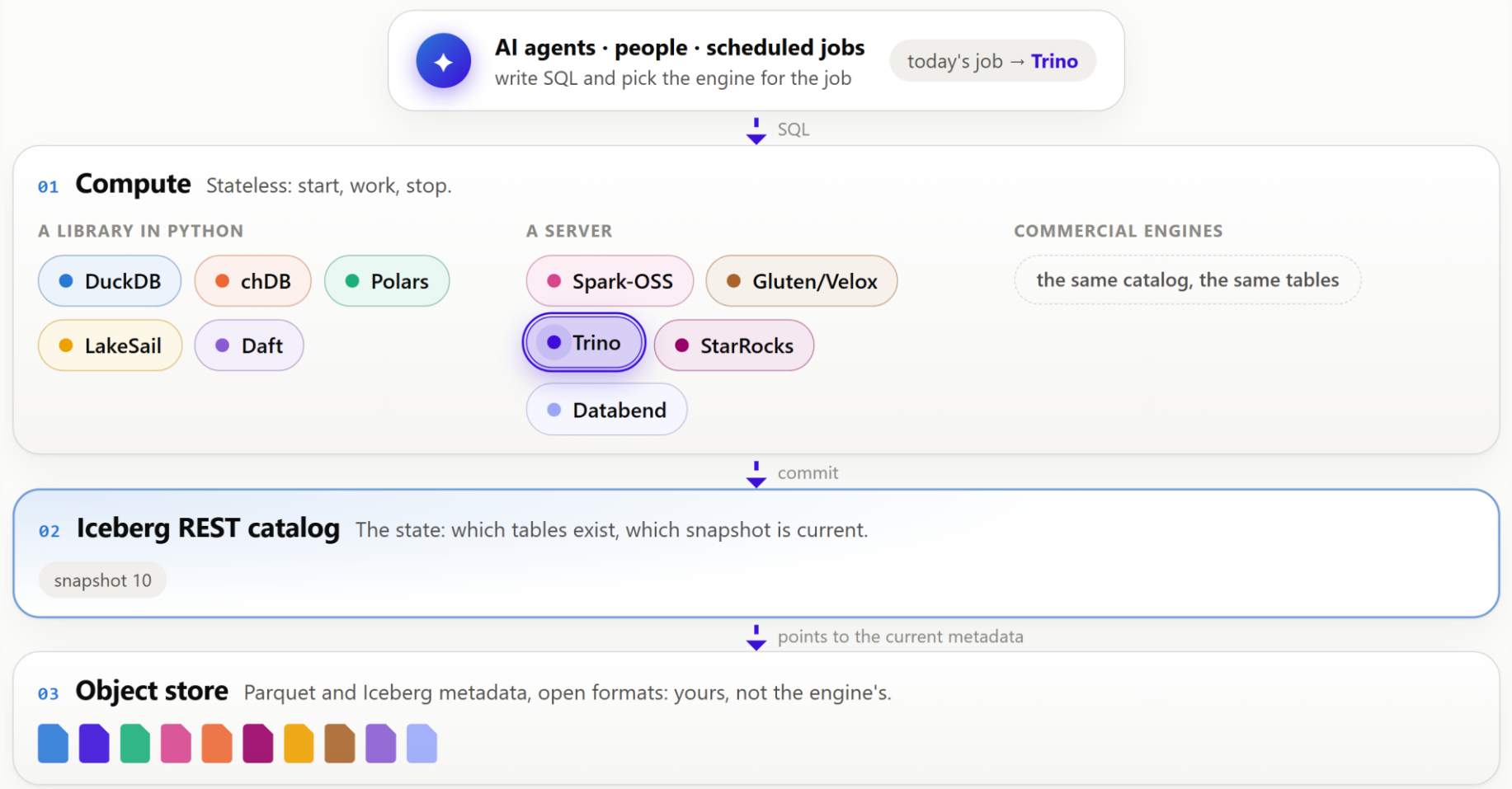


Before 1886, freight crossing North to South was unloaded and reloaded, or each car was lifted onto new wheels.

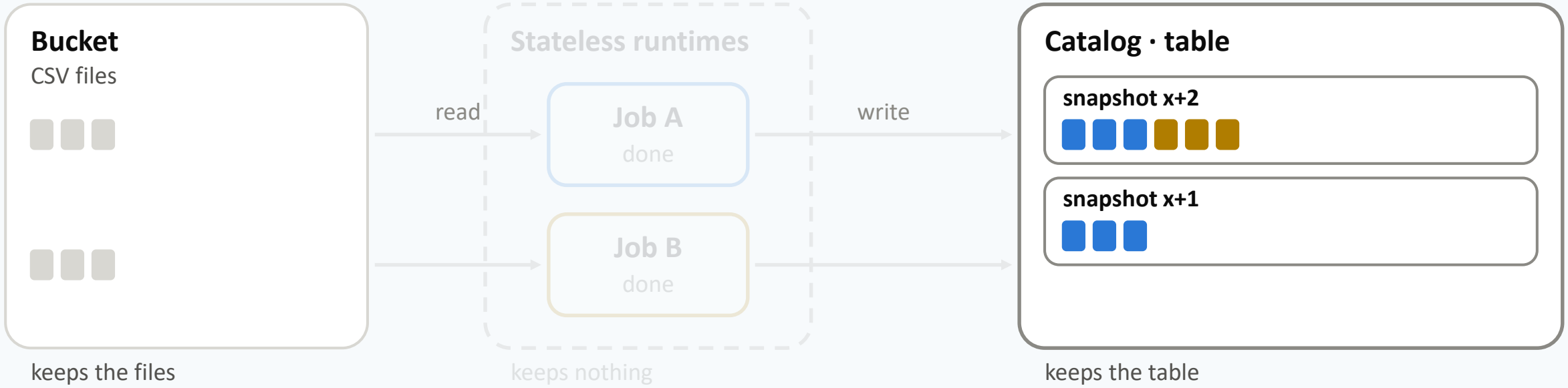
31 May – 1 June 1886: the South moved one rail on more than 11,000 miles of track, in 36 hours.

The trains were fine. The track wasn’t shared.

The open lakehouse



The catalog is the referee



Both jobs stop. Snapshot x+2 has the files of both jobs. If both jobs had changed the same files, the retry would fail.

03

The engines

16 engines tried, one catalog, one small box.

Disclosure: what this runs on

Compute

Free GitHub runners

4 cores · 16 GB

~105 GB free disk

Catalog

OneLake Catalog

Free through my job :)

Who they are

ENGINE	STARTED	WRITTEN IN	RUNS AS
Spark-OSS	UC Berkeley AMPLab, 2009	Scala	Python drives a local JVM
Gluten/Velox	Intel and Kyligence, 2022; Velox by Meta	C++, Scala	Python drives a local JVM
Trino	Facebook as Presto, 2012; Trino since 2020	Java	Client-server
StarRocks	Fork of Apache Doris, 2020	C++, Java	Client-server
Databend	Datafuse Labs, 2020	Rust	Client-server
DuckDB	CWI Amsterdam, 2019	C++	In-process
chDB	Auxten Wang, 2023	C++	In-process
Polars	Ritchie Vink, 2020	Rust	In-process
LakeSail	LakeSail, 2024	Rust	Client-server, Spark Connect
Daft	Eventual, 2022	Rust	In-process
Pandas	Wes McKinney, 2008	Python, C	In-process

Why DataFusion isn’t here

It’s everywhere: LakeSail and DataFusion Comet are built on it.

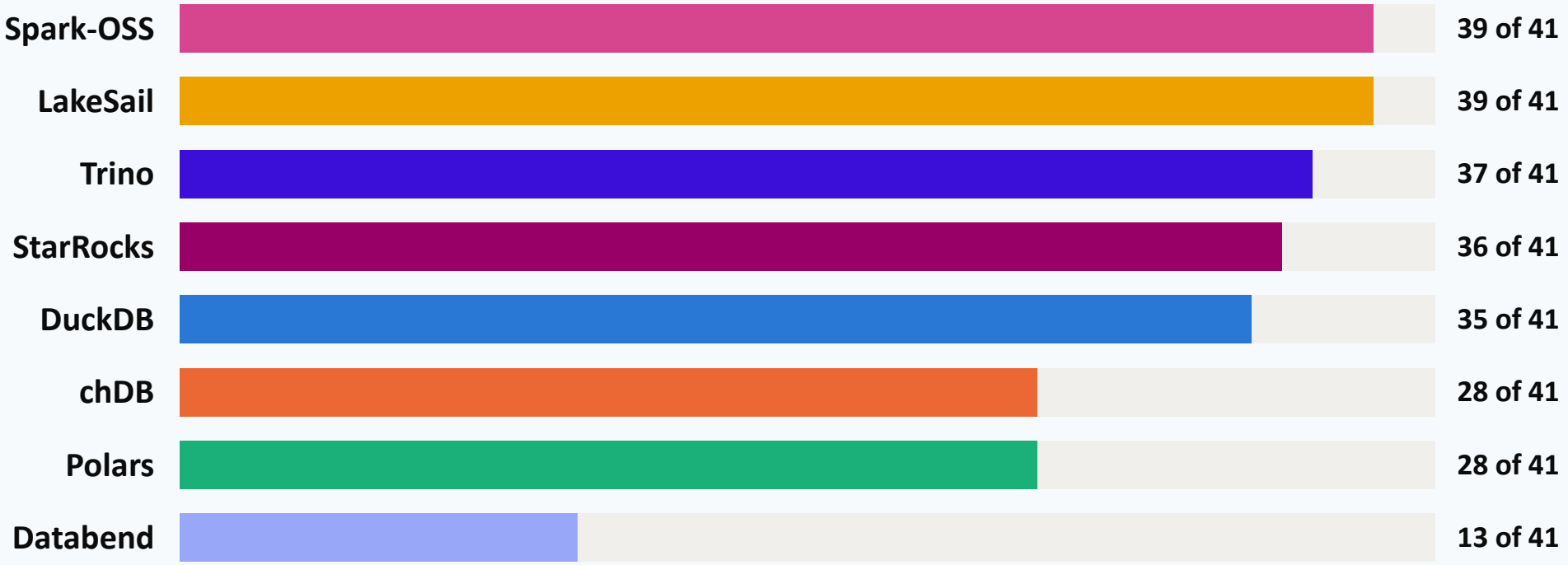
The Python package has no Iceberg: no catalog, no scan, no write.

Iceberg support is a separate repo, apache/datafusion-iceberg: Rust only, no release yet.

datafusion-python#1097

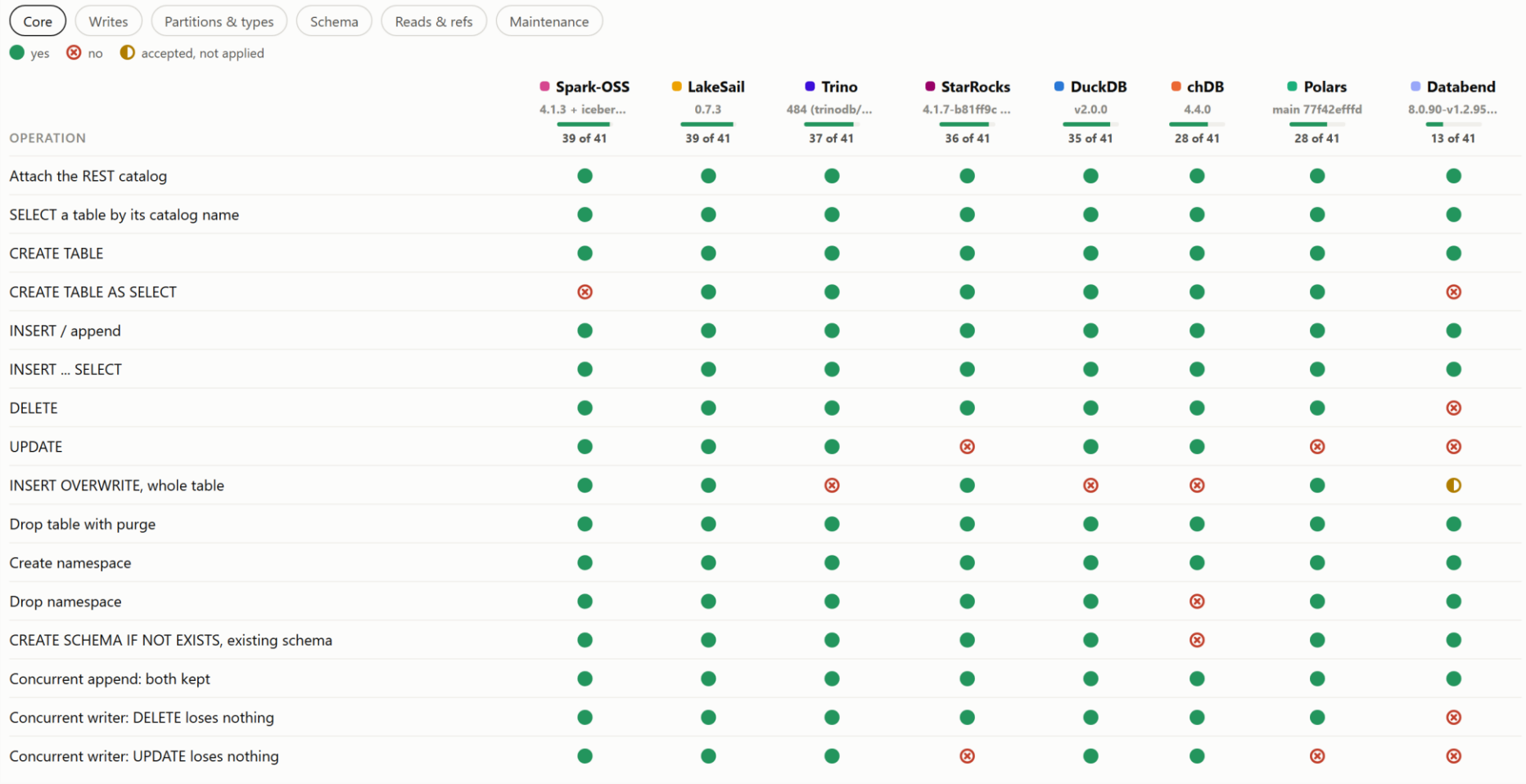
Iceberg support, counted

Operations each engine does, out of the 41 on the grid.



Polars: 6 on 25 Oct (2.1.0), 28 now (main). StarRocks joined at 36.

The Iceberg support grid



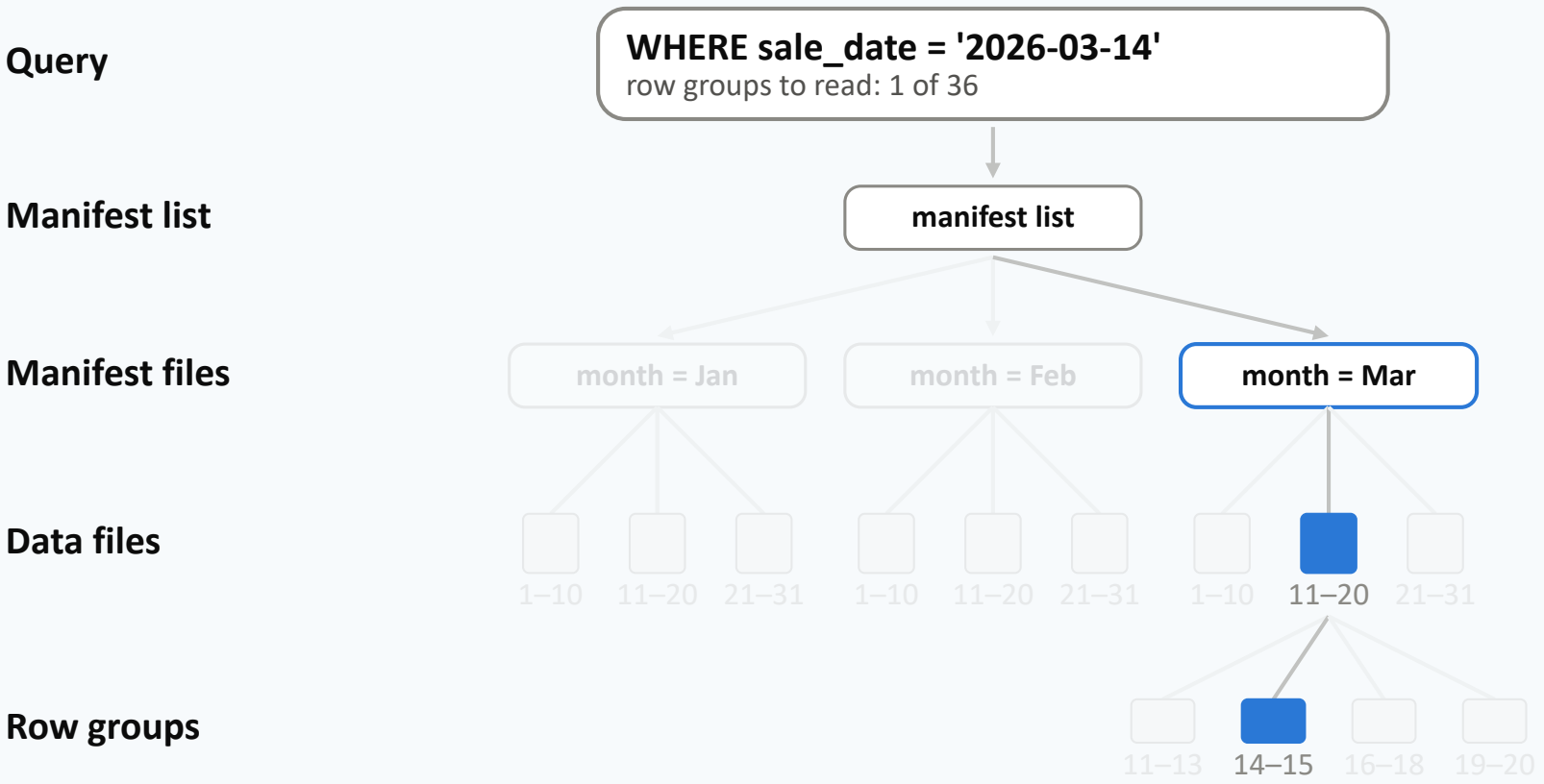
OPERATION	Spark-OSS 4.1.3 + iceber... 39 of 41	LakeSail 0.7.3 39 of 41	Trino 484 (trinodb/... 37 of 41	StarRocks 4.1.7-b81ff9c ... 36 of 41	DuckDB v2.0.0 35 of 41	chDB 4.4.0 28 of 41	Polars main 77f42efffd 28 of 41	Databend 8.0.90-v1.2.95... 13 of 41
Attach the REST catalog	●	●	●	●	●	●	●	●
SELECT a table by its catalog name	●	●	●	●	●	●	●	●
CREATE TABLE	●	●	●	●	●	●	●	●
CREATE TABLE AS SELECT	⊗	●	●	●	●	●	●	⊗
INSERT / append	●	●	●	●	●	●	●	●
INSERT ... SELECT	●	●	●	●	●	●	●	●
DELETE	●	●	●	●	●	●	●	⊗
UPDATE	●	●	●	⊗	●	●	⊗	⊗
INSERT OVERWRITE, whole table	●	●	⊗	●	⊗	⊗	●	◐
Drop table with purge	●	●	●	●	●	●	●	●
Create namespace	●	●	●	●	●	●	●	●
Drop namespace	●	●	●	●	●	⊗	●	●
CREATE SCHEMA IF NOT EXISTS, existing schema	●	●	●	●	●	⊗	●	●
Concurrent append: both kept	●	●	●	●	●	●	●	●
Concurrent writer: DELETE loses nothing	●	●	●	●	●	●	●	⊗
Concurrent writer: UPDATE loses nothing	●	●	●	⊗	●	●	⊗	⊗

04

What does “fast” mean?

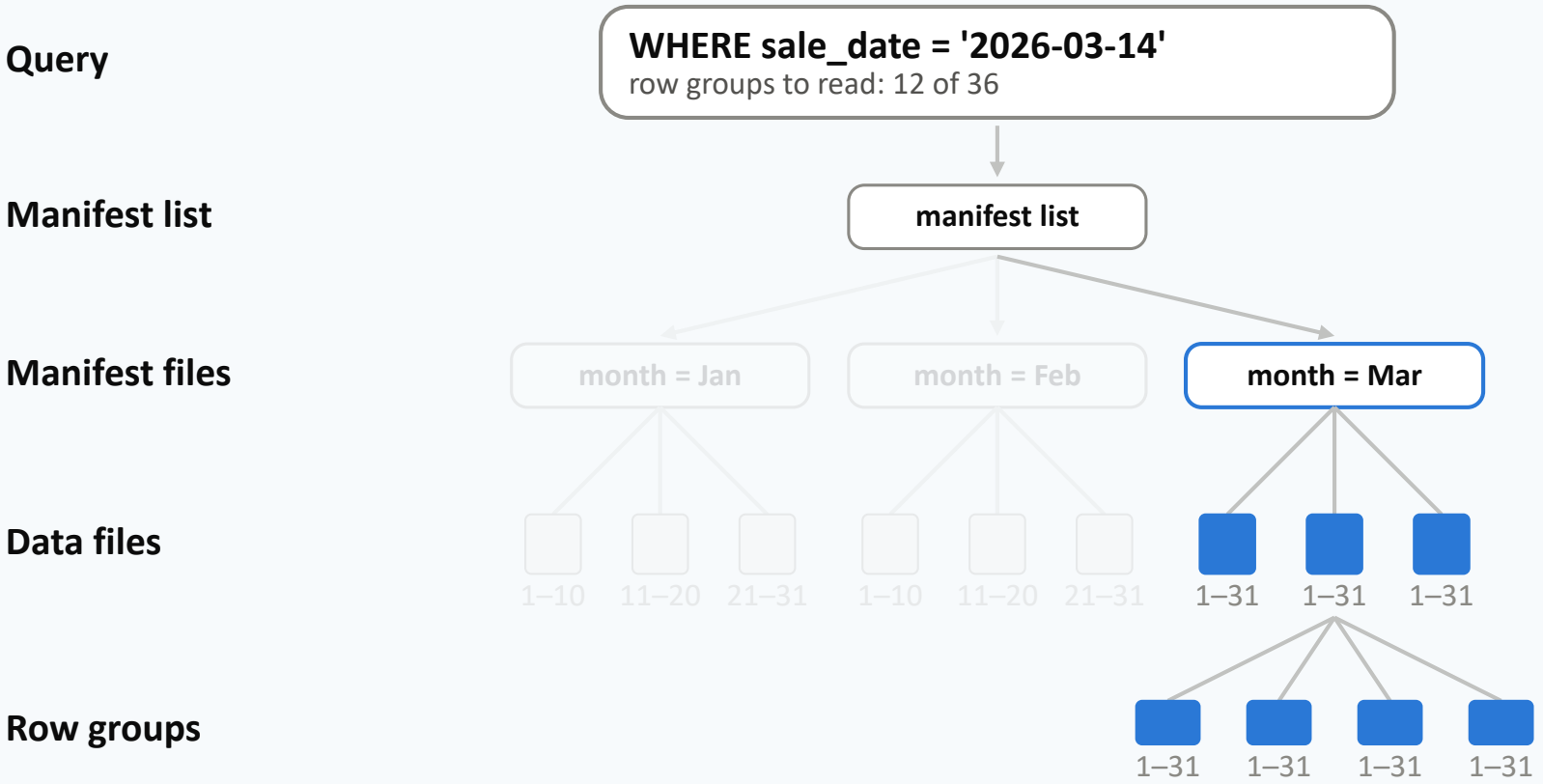
The best query is no query.

File skipping



1 row group of 36, before a single row was read.

The importance of sorting data



Random order: every March file spans days 1–31, so 12 row groups instead of 1.

The small-file problem

The same 1 GB of data, written two ways.

8 files of 128 MB

8 opens, 8 footers,
8 manifest entries.

8,000 files of 128 KB

8,000 opens, 8,000 footers,
8,000 manifest entries.

Every file costs a request to storage before a single row is read.

Compact into a few big files. Iceberg’s default target is 512 MB.

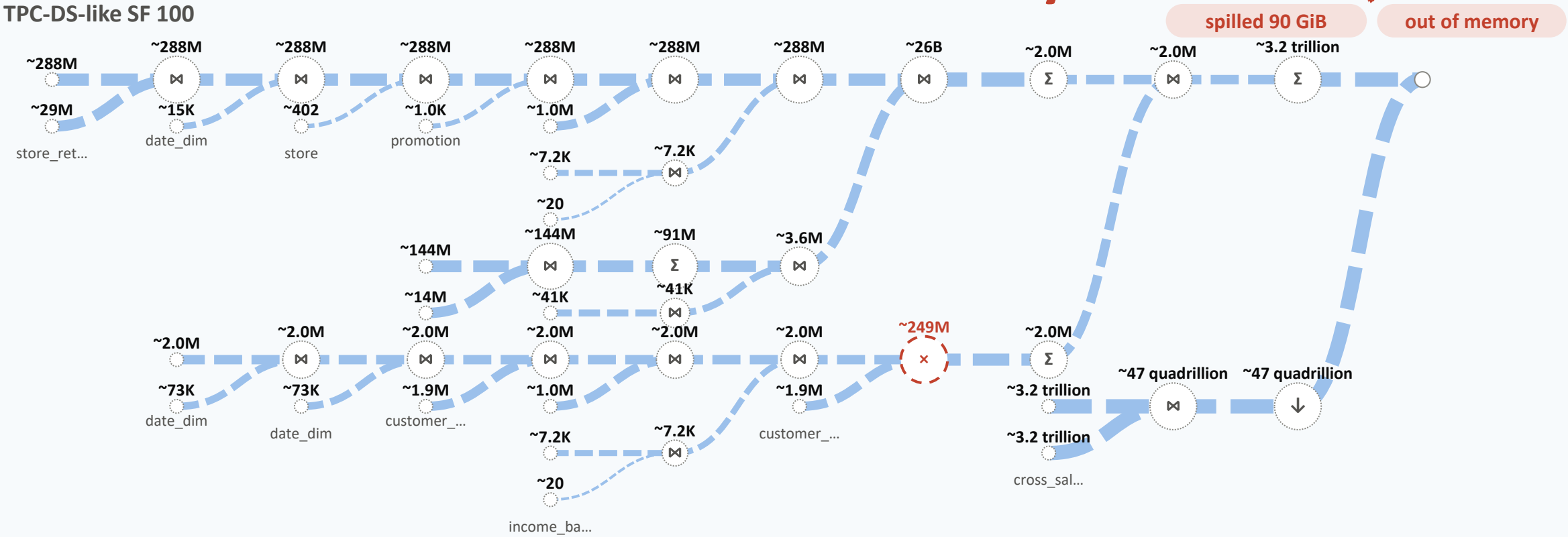
Q64 on DuckDB 2.0: a bad guess

DuckDB 2.0

TPC-DS-like SF 100

Out of memory

✗ 247 s



DuckDB 2.0 guesses a ≠ join makes 249M rows and runs it early, on its own. It makes about 3 trillion.

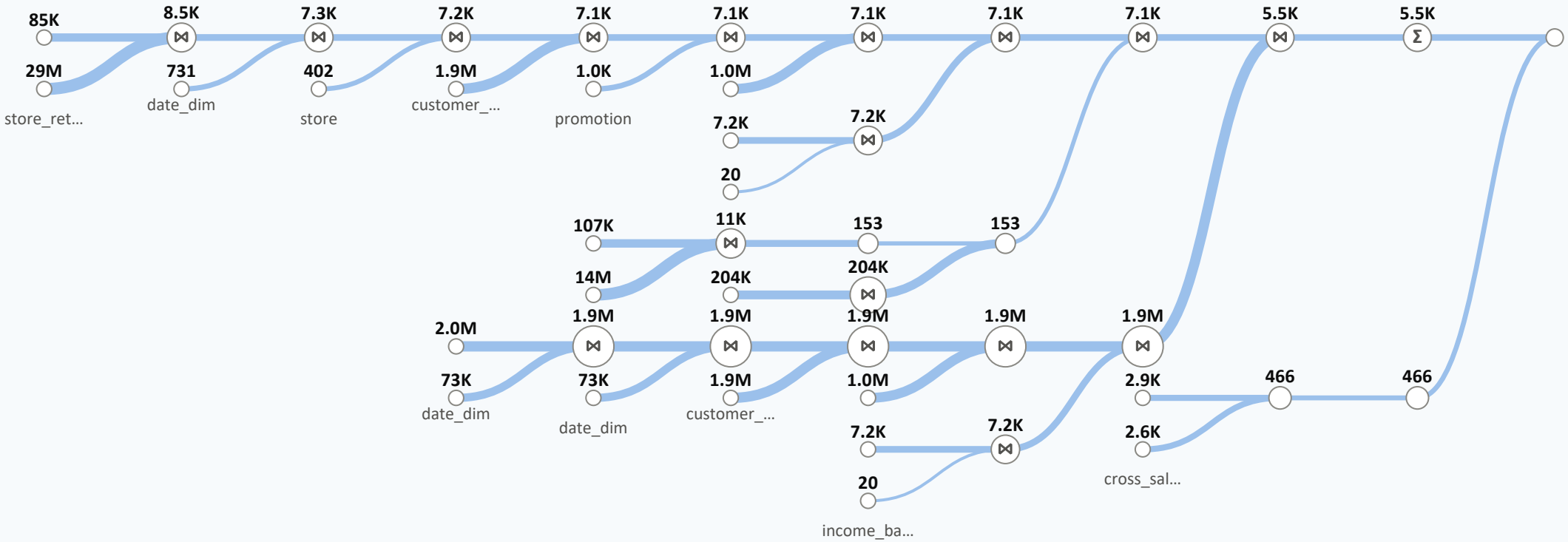
Q64 on the DuckDB nightly: fixed upstream

DuckDB nightly

✓ 101 s

done

TPC-DS-like SF 100 · peak memory **8.0 GiB** · largest intermediate result **1.9M** rows

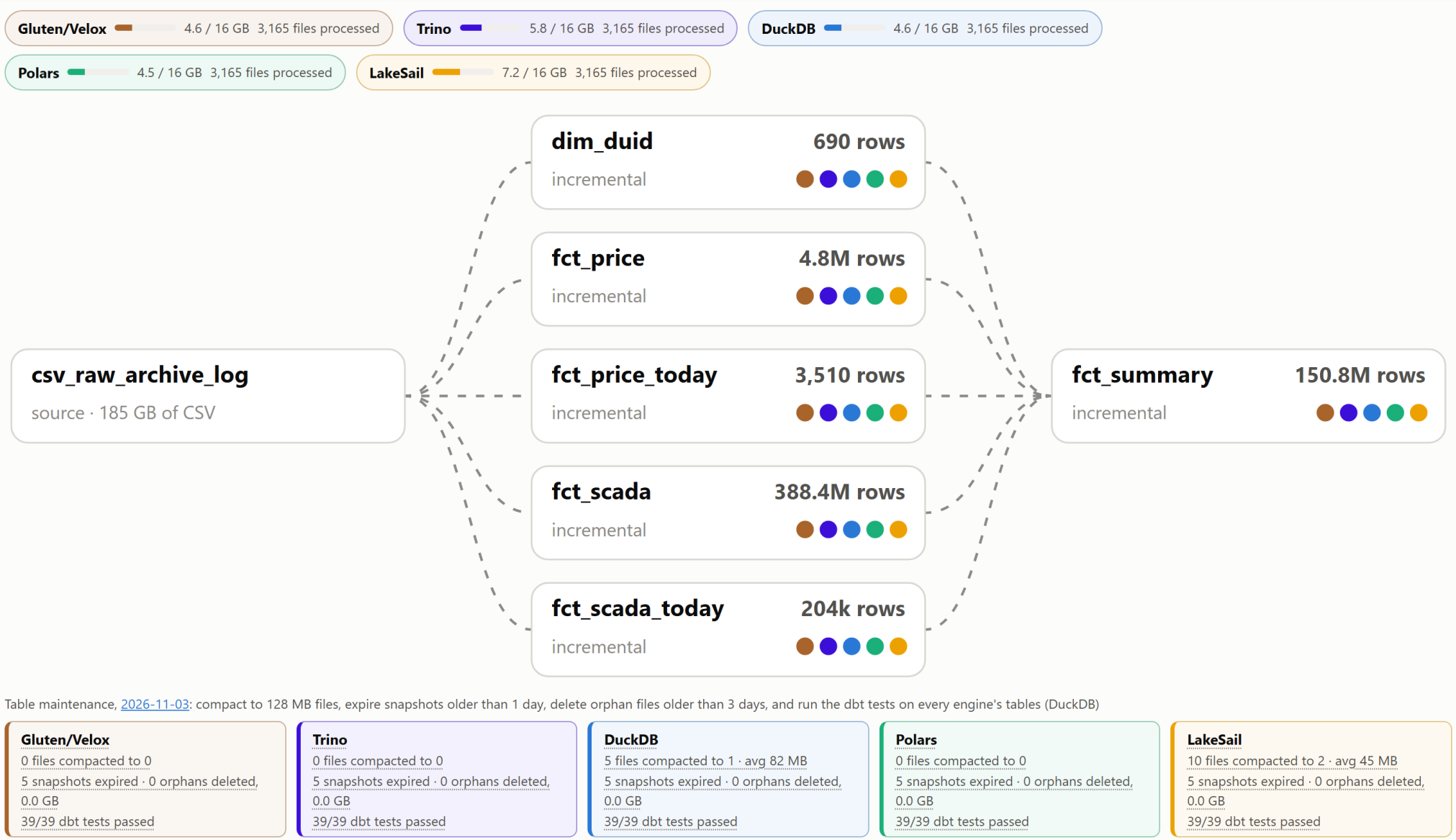


duckdb#27493 prices ≠ as nearly a cross product, so it is checked inside a hash join. Done in 101 s, no step above 1.9M rows.

05

Real-life use case

The only one that matters.


Gluten/Velox 4.6 / 16 GB 3,165 files processed
Trino 5.8 / 16 GB 3,165 files processed
DuckDB 4.6 / 16 GB 3,165 files processed
Polars 4.5 / 16 GB 3,165 files processed
LakeSail 7.2 / 16 GB 3,165 files processed
csv_raw_archive_log
source · 185 GB of CSV
dim_duid
690 rows
incremental
fct_price
4.8M rows
incremental
fct_price_today
3,510 rows
incremental
fct_scada
388.4M rows
incremental
fct_scada_today
204k rows
incremental
fct_summary
150.8M rows
incremental
Table maintenance, 2026-11-03: compact to 128 MB files, expire snapshots older than 1 day, delete orphan files older than 3 days, and run the dbt tests on every engine's tables (DuckDB)
Gluten/Velox
0 files compacted to 0
5 snapshots expired · 0 orphans deleted, 0.0 GB
39/39 dbt tests passed
Trino
0 files compacted to 0
5 snapshots expired · 0 orphans deleted, 0.0 GB
39/39 dbt tests passed
DuckDB
5 files compacted to 1 · avg 82 MB
5 snapshots expired · 0 orphans deleted, 0.0 GB
39/39 dbt tests passed
Polars
0 files compacted to 0
5 snapshots expired · 0 orphans deleted, 0.0 GB
39/39 dbt tests passed
LakeSail
10 files compacted to 2 · avg 45 MB
5 snapshots expired · 0 orphans deleted, 0.0 GB
39/39 dbt tests passed

Same code, two engines

LakeSail

Rust · DataFusion · Spark Connect

```
sail:
  type: spark
  method: session
  schema: mart
  threads: 1
```

203s · 6 of 6 models · dbt-spark 1.11.0

models/aemo/spark/

6 models, one copy

Same Iceberg tables, same catalog

Spark + Gluten/Velox

JVM · C++ underneath

```
gluten:
  type: spark
  method: session
  schema: mart
  threads: 1
```

327s · 6 of 6 models · dbt-spark 1.11.0

dbt + Iceberg: a new way to do data engineering.

The code and the tables stay. The engine is a target in profiles.yml.

Why real life is hard

A minor feature can be a blocker

A ragged CSV: 53 fields on one row, about 120 on the next. If the reader can’t take it, nothing runs.

Append only works in a demo

Runs overlap. Two appends of the same files both commit, and the rows land twice. A MERGE on the key keeps one copy.

Spill to disk is the real bottleneck

The merge doesn’t fit in 16 GB of RAM, so it goes to disk: up to 3 GB in one run. No spill, and the runner dies.

The adapter matters as much as the engine

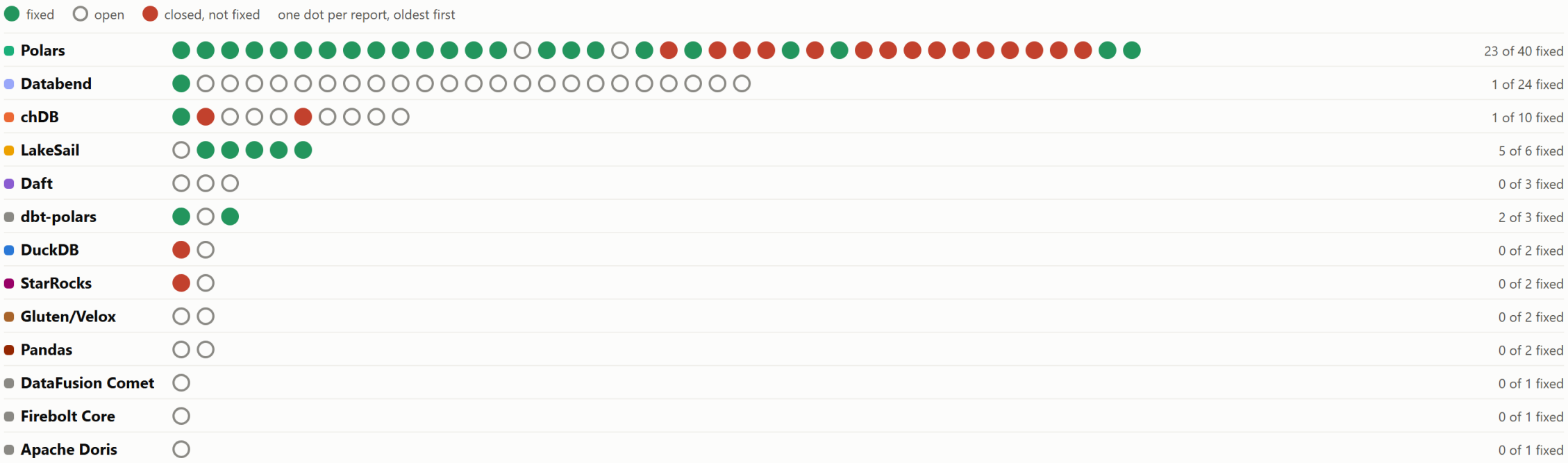
dbt decides how the batch is staged: a temp table or a view. Same SQL, very different memory.

Every one of these can stop an engine, whatever its benchmark says.

Every report, one dot each

The best help for an engine is a bug report it can reproduce. No PR needed: the maintainers fix it, often within days.

97	32	5.5 days	4
bug reports filed upstream	fixed	median, from report to fix	pull requests from the bench



What makes a report land: one script on a clean machine · the smallest case · expected and actual · the version, and main · one bug per issue (Daft #7532 became #7601 and #7602).

06

What I learned

Read less

On a small box, bytes off storage cost more than CPU.

Cache your reads

370 s → 92 s

DuckDB, TPC-DS-like SF10 cold: file cache off, then on.

Filter on a date, not a file name

failed → 168 s

Sail's merge. Iceberg cuts string bounds to 16 characters; dates are never cut.

Distributed engines run fine on one small box

Each on one 4-core, 16 GB runner.

StarRocks · TPC-H-like SF300

3,943s

A Java front end and a C++ back end, sharing one VM.

Spark, Velox underneath · TPC-DS-like SF100

5,762s

3.7× faster than Spark's own engine, same SQL. Never ran out of memory here.

Engine execution nuances

One value, 96.07385, cast from DOUBLE to DECIMAL(18,4)

Spark

96.0739

DuckDB

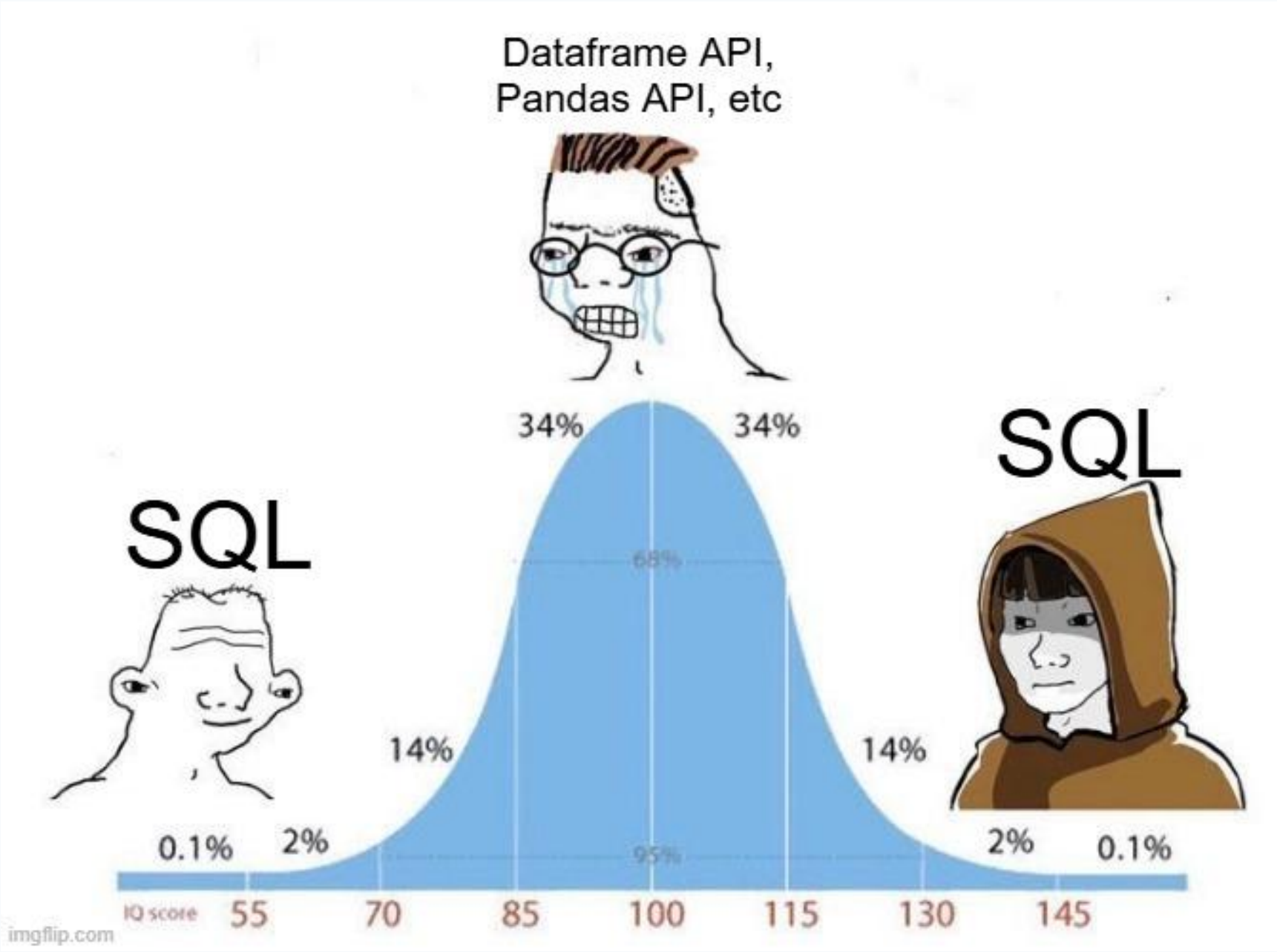
96.0738

Out of 3.08M rows: DuckDB differs from Spark on 6,716, Polars on 85,090. Each by 0.0001.

Same SQL, not the same behaviour. The fix: DECIMAL(18,5), nothing rounds.

My use case used to work with only 1 engine. Now we have 5. That's progress.




Dataframe API,
Pandas API, etc
SQL
SQL
34%
34%
14%
14%
0.1%
2%
2%
0.1%
68%
95%
IQ score
55
70
85
100
115
130
145
imgflip.com

Thank you

Questions? · djouallah.github.io/lakehouse_benchmark



Scan for the code:
github.com/djouallah/lakehouse_benchmark